VERA RBL-XE

FPGA Build & Programming Guide

iCE40UP5K – Rev 3

6502 bus controller with RAMbo 256 KB, PBI RAM/ROM,
and external SRAM support

Last updated: June 11, 2026

0 Contents

1 Overview

This document describes the complete workflow for building and programming the FPGA firmware for the VERA RBL-XE module. The firmware implements a 6502 bus controller on a Lattice iCE40UP5K, replacing the ESP32-S3 MCU used in the previous design.

1.1 Architecture summary

- **PBI RAM** (`$D600`–`$D7FF`, 512 B): stored in iCE40 BRAM (1 EBR block).
- **PBI ROM** (`$D800`–`$DFFF`, 2 KB): stored in iCE40 BRAM (4 EBR blocks), pre-loaded at synthesis time from the assembled 6502 driver binary.
- **RAMbo** (`$4000`–`$7FFF`, 16 KB window, 256 KB total): served by an external 512K×8 SRAM (e.g. AS6C4008-55SIN). The FPGA controls chip-enable, output-enable, write-enable, and the upper four address bits (bank), using only 7 new I/O pins.
- **PHI2 synchronisation**: PHI2 is treated as a data input and synchronised to the 25 MHz FPGA clock via a two-stage flip-flop chain, producing a single-cycle `phi2_rise` pulse used to trigger all registered logic.
- **Two bitstreams** – `VERA_IS_PBI` **generic**: the top-level entity exposes a VHDL-2008 boolean generic that selects the bus interface at synthesis time. *PBI mode* (`VERA_IS_PBI=true`): latch at `$D1FF`, PBI RAM and ROM in BRAM (5 EBR blocks), EXTSEL for PBI RAM and RAMbo, MPD for PBI ROM reads. *CCTL mode* (`VERA_IS_PBI=false`): latch at `$D5FF` (write `$80` to select, `$00` to deselect), no BRAM (Yosys eliminates dead code: 0 EBR), EXTSEL for RAMbo only, MPD always inactive.
- **Hardware configuration pins**: `PBI_ID_SEL` (pin 27) is wired via PCB jumper to one of D[0..7]; the sampled bit value latches VERA_CS in PBI mode (`vera_cs_latch <= not PBI_ID_SEL`). `RAMBO_EN` (pin 28) with pullup (VCC) enables RAMbo; with pulldown (GND) disables it regardless of PBCTL[2].

1.2 Memory map

Range	Size	Function	Implementation
`$4000`–`$7FFF`	16 KB window	RAMbo (256 KB banked)	External SRAM
`$D1FF`	1 B	VERA_CS latch (write)	Register
`$D301`	1 B	PIA PORTB snoop	Register
`$D303`	1 B	PIA PBCTL snoop	Register
`$D600`–`$D7FF`	512 B	PBI RAM	BRAM (1 EBR)
`$D800`–`$DFFF`	2 KB	PBI ROM	BRAM (4 EBR)

Note (CCTL mode): when built with `VERA_IS_PBI=false` the VERA_CS latch address changes to `$D5FF` (write `$80` to select, `$00` to deselect), and PBI RAM/ROM are absent (0 EBR).

2 Prerequisites

2.1 FPGA toolchain

All tools below are open-source. Install on Debian/Ubuntu:

```
sudo apt install fpga-icestorm nextpnr-ice40 yosys ghdl
```

Note: the `ghdl-yosys-plugin` is required for VHDL synthesis and is generally not available as a package. Build it from source:

```
git clone https://github.com/ghdl/ghdl-yosys-plugin
```

```
cd ghdl-yosys-plugin
make
sudo make install
```

2.2 Flash programming tools

```
# openFPGALoader -- recommended, supports many programmers
sudo apt install openFPGALoader

# iceprog is already included in fpga-icestorm
```

2.3 6502 assembler (cc65)

The PBI ROM driver is written in 6502 assembly and built with the cc65 toolchain.

```
sudo apt install cc65
```

2.4 Python 3

Required to run `gen_pbi_rom_pkg.py`, which converts the assembled 6502 binary into a VHDL constant package.

```
sudo apt install python3
```

2.5 Tool roles at a glance

Tool	Input	Output	Purpose
`ca65` / `ld65`	`.s` sources	`.bin`	6502 assembly
`gen_pbi_rom_pkg.py`	`.bin`	`.vhd`	ROM initialisation
`ghdl` (plugin)	VHDL sources	RTLIL	VHDL elaboration
`yosys`	RTLIL	`.json`	Synthesis
`nextpnr-ice40`	`.json + .pcf`	`.asc`	Place & route
`icepack`	`.asc`	`.bin`	Bitstream packing
`openFPGALoader`	`.bin`	SPI flash	Programming

3 Build Flow

3.1 Step 1 – Assemble the 6502 ROM driver

The PBI ROM at `$D800`–`$DFFF` contains a 2 KB 6502 driver assembled from `firmware/MCU/6502/src/vera_pbi_`

```
cd firmware/MCU/6502
make
```

Listing 1: Assemble the 6502 driver

This single command runs three sub-steps:

1. **Assemble**: `ca65` compiles `vera_pbi_handler.s` into `vera_pbi_handler.o`.
2. **Link**: `ld65` links with `pbi-driver.ld` to produce `vera_pbi_handler.bin`.
3. **Generate VHDL ROM package**: `gen_pbi_rom_pkg.py` reads the binary and writes `../../FPGA/pbi_rom_pkg.vhd` with the `PBI_ROM_INIT` constant.

To regenerate only the VHDL package when the binary already exists:

```
make fpga-rom
```

3.2 Step 2 – Synthesis

The top-level entity exposes a VHDL-2008 boolean generic, `VERA_IS_PBI` (default `true`), that selects the bus interface at synthesis time. The Makefile invokes Yosys twice, producing two independent bitstreams from the same source:

```
cd firmware/FPGA
yosys -m ghdl -p \
    "ghdl --std=08 -gVERA_IS_PBI=true \
          custom_types.vhd pbi_rom_pkg.vhd C6502_Interface.vhd \
          -e C6502_Interface; \
     synth_ice40 -top C6502_Interface -json C6502_Interface_PBI.json"
```

Listing 2: PBI synthesis (VERA_IS_PBI=true)

```
yosys -m ghdl -p \
    "ghdl --std=08 -gVERA_IS_PBI=false \
          custom_types.vhd pbi_rom_pkg.vhd C6502_Interface.vhd \
          -e C6502_Interface; \
     synth_ice40 -top C6502_Interface -json C6502_Interface_CCTL.json"
```

Listing 3: CCTL synthesis (VERA_IS_PBI=false)

The `-m ghdl` flag loads the GHDL plugin into Yosys, which handles VHDL analysis and elaboration. When `VERA_IS_PBI=false`, Yosys eliminates all PBI RAM/ROM dead code, reducing resource usage dramatically. Measured values after place-and-route with nextpnr-ice40:

Resource	**PBI**	**CCTL**	**Available**	**% (PBI)**
ICESTORM_LC (total)	87	1	5280	1%
of which LUT4 only	52	1		
of which LUT4+DFF	5	0		
of which DFF only	5	0		
of which CARRY only	21	0		
ICESTORM_RAM (EBR)	5	0	30	16%
SB_IO (I/O buffers)	39	39	39	100%
ICESTORM_SPRAM	0	0	4	0%
Fmax (CLK)	**62.72 MHz** (PBI, PASS at 25 MHz)			

I/O budget note: the iCE40UP5K-SG48 exposes exactly 39 user I/O pins, and this design uses all 39. The mode selection (PBI vs. CCTL) is compiled into the bitstream via `VERA_IS_PBI` rather than being a runtime input, freeing the two former `DIP_SEL` pins for dedicated hardware functions: `PBI_ID_SEL` (pin 27) and `RAMBO_EN` (pin 28).

3.3 Step 3 – Place & Route

Place & route assigns logic cells and routes signals to physical iCE40 resources, guided by the pin constraints file.

```
nextpnr-ice40 --up5k --package sg48 \
    --json C6502_Interface_PBI.json \
```

```
    --pcf board-pin-mapping.pcf \
    --asc C6502_Interface_PBI.asc
```

Listing 4: Place & route with nextpnr-ice40 (PBI example)

The CCTL build uses the same command with `_CCTL` filename suffixes.

Important: all pin numbers in `board-pin-mapping.pcf` are placeholders. Edit the file to match your actual schematic before this step (see Section 5).

nextpnr reports timing after routing. Measured results:

Path type	Description	Worst delay
CLK → CLK	Registered path (Fmax)	15.94 ns (62.72 MHz)
async → async	Combinational (PHI2 gate)	28.12 ns
async → CLK	Input setup time	26.89 ns
CLK → async	Clock-to-output	17.17 ns

The 25 MHz system clock (period 40 ns) has **24.06 ns of slack** on the registered path. The combinational PHI2-gated outputs settle in at most 28.12 ns after address/control inputs change — well within the 6502 PHI2-high window of ≈279 ns at 1.79 MHz.

3.4 Step 4 – Pack the bitstream

`icepack` converts the ASCII intermediate format produced by nextpnr into the binary bitstream that is written to the SPI flash.

```
icepack C6502_Interface_PBI.asc  C6502_Interface_PBI.bin
icepack C6502_Interface_CCTL.asc C6502_Interface_CCTL.bin
```

Listing 5: Pack binary bitstreams

The `.bin` file is what gets programmed into the SPI flash. Bitstream size is device-dependent (not design-dependent) on the iCE40: both bitstreams measure **≈104 KB** (iCE40UP5K SG48).

3.5 Step 5 – Program the SPI flash

The iCE40UP5K loads its configuration from an external SPI NOR flash at power-up. The programmer writes the bitstream into that flash.

3.5.1 Using openFPGALoader (recommended)

```
# PBI bitstream (PBI connector, $D1xx)
openFPGALoader -b ice40_generic C6502_Interface_PBI.bin
# CCTL bitstream (cartridge port, $D5xx)
openFPGALoader -b ice40_generic C6502_Interface_CCTL.bin
```

Listing 6: Program with openFPGALoader

Select the programmer cable explicitly if needed:

```
openFPGALoader --cable ft2232      C6502_Interface_PBI.bin   # FTDI
   FT2232H
openFPGALoader --cable ft232       C6502_Interface_PBI.bin   # FTDI
   FT232H
openFPGALoader --cable tigard      C6502_Interface_PBI.bin   # Tigard
```

```
openFPGALoader --cable raspberrypi C6502_Interface_PBI.bin   #
   Raspberry Pi GPIO
```

3.5.2 Using iceprog (FTDI FT2232H)

```
iceprog C6502_Interface_PBI.bin    # PBI bitstream
iceprog C6502_Interface_CCTL.bin   # CCTL bitstream

# Erase + write (useful if previous write was partial)
iceprog -e 1048576 C6502_Interface_PBI.bin
```

Listing 7: Program with iceprog

3.6 One-command build and flash

The `firmware/FPGA/Makefile` wraps all steps:

```
cd firmware/FPGA
make pbi                # synthesise + P&R + pack ->
   C6502_Interface_PBI.bin
make cctl               # synthesise + P&R + pack ->
   C6502_Interface_CCTL.bin
make                    # build both (default)
make flash-pbi          # write PBI  bitstream via openFPGALoader
make flash-cctl         # write CCTL bitstream via openFPGALoader
make flash-pbi-iceprog   # write PBI  via iceprog (FTDI FT2232H)
make flash-cctl-iceprog  # write CCTL via iceprog
make doc                # regenerate PDF documentation
make clean              # remove all generated files
```

4 Programmer Wiring

The SPI flash is shared between the FPGA (configuration bus at boot) and the programmer (writes the bitstream). During programming, the FPGA is held in reset by asserting `CRESET_B` low.

Programmer signal	iCE40UP5K pin	SPI flash pin
MOSI	SPI_SI	DI
MISO	SPI_SO	DO
SCK	SPI_SCK	CLK
CS#	–	CS#
CRST#	CRESET_B	–
CDONE	CDONE	–

Recommended SPI flash: Winbond W25Q16 (2 MB) or W25Q32 (4 MB).

5 Pin Assignment

Edit `board-pin-mapping.pcf` and replace every pin number with the actual pad number from your schematic before running nextpnr-ice40.

The `CLK` signal must be assigned to a **Global Buffer (GB) input** pin. Valid GB pins on the iCE40UP5K SG48 package are: **20, 28, 31, 38, 41, 43, 48**.

Valid user I/O pin numbers for SG48 (39 total):

```
2, 3, 4, 6, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,
23, 25, 26, 27, 28, 31, 32, 34, 35, 36, 37, 38, 39,
40, 41, 42, 43, 44, 45, 46, 47, 48
```

Pins 1, 5, 7, 8, 22, 24, 29, 30, 33 are power/ground or reserved and **must not** appear in the PCF file — nextpnr rejects them with an error.

5.1 Complete pin table

Signal	Dir	PCF pin	Std	Description
CLK	In	35*	LVCMOS33	25 MHz system clock. Must land on a GB pin.
A[0]	In	40*	LVCMOS33	6502 address bit 0
A[1]	In	41*	LVCMOS33	6502 address bit 1
A[2]	In	42*	LVCMOS33	6502 address bit 2
A[3]	In	43*	LVCMOS33	6502 address bit 3
A[4]	In	44*	LVCMOS33	6502 address bit 4
A[5]	In	45*	LVCMOS33	6502 address bit 5
A[6]	In	46*	LVCMOS33	6502 address bit 6
A[7]	In	47*	LVCMOS33	6502 address bit 7
A[8]	In	48*	LVCMOS33	6502 address bit 8
A[9]	In	6*	LVCMOS33	6502 address bit 9
A[10]	In	2*	LVCMOS33	6502 address bit 10
A[11]	In	3*	LVCMOS33	6502 address bit 11
A[12]	In	4*	LVCMOS33	6502 address bit 12
A[13]	In	26*	LVCMOS33	6502 address bit 13. Also wired to SRAM A[13] on PCB.
A[14]	In	9*	LVCMOS33	6502 address bit 14
A[15]	In	10*	LVCMOS33	6502 address bit 15
D[0]	InOut	11*	LVCMOS33	6502 data bit 0. Shared net with SRAM DQ[0].
D[1]	InOut	12*	LVCMOS33	6502 data bit 1. Shared net with SRAM DQ[1].
D[2]	InOut	13*	LVCMOS33	6502 data bit 2. Shared net with SRAM DQ[2].
D[3]	InOut	14*	LVCMOS33	6502 data bit 3. Shared net with SRAM DQ[3].
D[4]	InOut	15*	LVCMOS33	6502 data bit 4. Shared net with SRAM DQ[4].
D[5]	InOut	16*	LVCMOS33	6502 data bit 5. Shared net with SRAM DQ[5].
D[6]	InOut	17*	LVCMOS33	6502 data bit 6. Shared net with SRAM DQ[6].
D[7]	InOut	18*	LVCMOS33	6502 data bit 7. Shared net with SRAM DQ[7].
PHI2	In	34*	LVCMOS33	6502 phase-2 clock. Synchronised internally; not used as FPGA clock.
RNW	In	36*	LVCMOS33	Read/Not-Write. High = read, Low = write.
VERA_CS	Out	32*	LVCMOS33	VERA chip-select latch output (active low).

continued on next page

Signal	Dir	PCF pin	Std	Description
MPD	Out	31*	LVCMOS33	Machine Program Data (active low, **PBI mode only**). Asserted during PBI ROM ($D800–$DFFF) reads; blocks Atari from driving D[7:0]. Always inactive (high) in CCTL mode.
EXTSEL	Out	37*	LVCMOS33	External Select (active low). **PBI mode:** asserted for PBI RAM ($D600–$D7FF) and RAMbo ($4000–$7FFF). **CCTL mode:** asserted for RAMbo only. Blocks Atari internal RAM from responding.
PBI_ID_SEL	In	27*	LVCMOS33	Hardware jumper to one of D[0..7] on PCB. Sampled when $D1FF is written (PBI mode only); `vera_cs_latch <= not PBI_ID_SEL`. Logic 1 on the chosen bit selects VERA, logic 0 deselects. Ignored in CCTL build.
RAMBO_EN	In	28*	LVCMOS33	Tie to VCC (pullup) to enable RAMbo hardware; tie to GND (pulldown) to disable RAMbo completely, overriding PBCTL[2]. Both RAMBO_EN and PBCTL[2] must be high for RAMbo to respond.
SRAM_A_BANK[0]	Out	19*	LVCMOS33	SRAM address bit 14 (bank LSB).
SRAM_A_BANK[1]	Out	20*	LVCMOS33	SRAM address bit 15.
SRAM_A_BANK[2]	Out	21*	LVCMOS33	SRAM address bit 16.
SRAM_A_BANK[3]	Out	38*	LVCMOS33	SRAM address bit 17 (bank MSB).
SRAM_CE_N	Out	23*	LVCMOS33	SRAM chip enable (active low). Asserted during PHI2-high when address is in $4000–$7FFF and RAMbo is enabled.
SRAM_OE_N	Out	39*	LVCMOS33	SRAM output enable (active low). Asserted on read cycles only.
SRAM_WE_N	Out	25*	LVCMOS33	SRAM write enable (active low). Asserted on write cycles only. Pulse width equals PHI2-high time (≈279 ns at 1.79 MHz).

** Pin numbers are placeholders. Replace with actual pad numbers from your board schematic.*

5.2 External SRAM address wiring

The SRAM address bus is split between PCB traces (lower 14 bits) and FPGA outputs (upper 4 bank bits):

SRAM address bit	Source	Note
A[13:0]	6502 A[13:0] (PCB net)	Direct trace, no FPGA I/O pin
A[14]	FPGA `SRAM_A_BANK[0]`	Bank bit 0 = PORTB[2]
A[15]	FPGA `SRAM_A_BANK[1]`	Bank bit 1 = PORTB[3]
A[16]	FPGA `SRAM_A_BANK[2]`	Bank bit 2 = PORTB[5]
A[17]	FPGA `SRAM_A_BANK[3]`	Bank bit 3 = PORTB[6]
A[18]	GND (PCB)	Selects lower 256 KB of 512 KB chip

5.3 RAMbo bank formula

The FPGA snoops writes to the Atari PIA to derive the active bank:

```
PORTB at $D301  -- bank bits
PBCTL at $D303  -- bit 2 enables RAMbo

bank[1:0] = PORTB[3:2]
bank[3:2] = PORTB[6:5]
=> SRAM A[17:14] = { PORTB[6], PORTB[5], PORTB[3], PORTB[2] }
```

RAMbo is active only when both `RAMBO_EN` (pin 28 tied to VCC) and `PBCTL[2] = 1` (software enable written by the Atari OS).

6 Timing

6.1 Read cycle (SRAM)

Event	Time from PHI2 rise	Note
Address valid (6502 tADS)	−40 ns	Before PHI2 rise
SRAM_CE# asserted	≈0 ns	Combinational gate on PHI2
SRAM data valid (tAA=55 ns)	+55 ns	AS6C4008-55
6502 data setup deadline	+249 ns	279 ns − 30 ns tDSR
Read margin	**+194 ns**	Adequate

6.2 Write cycle (SRAM)

WE# is asserted combinationally when PHI2 is high and the address is in the RAMbo window. The pulse width equals the PHI2-high time:

$$t_{WC} = t_{\text{PHI2-high}} \approx 279\,\text{ns} \quad \gg \quad t_{WC(\text{SRAM})} = 55\,\text{ns}$$

6.3 PBI RAM / ROM (BRAM)

BRAM reads are triggered on `phi2_rise` (one clock cycle after the real PHI2 edge, ≈40 ns delay). The BRAM output is registered and valid within one additional clock cycle (≈40 ns), leaving ≈199 ns of margin before the 6502 samples the bus.

7 Source File Reference

File	Description
`C6502_Interface.vhd`	Top-level VHDL entity (Rev 3). Generic `VERA_IS_PBI` selects PBI or CCTL mode at synthesis time. Contains all bus logic, BRAM instances, and SRAM control.
`custom_types.vhd`	Package defining `rom_array` (2048×8) and `ram_512_array` (512×8) array types.
`pbi_rom_pkg.vhd`	Auto-generated package containing the `PBI_ROM_INIT` constant (2048 bytes). Do not edit manually.
`gen_pbi_rom_pkg.py`	Python 3 script that converts a 6502 `.bin` file into `pbi_rom_pkg.vhd`. Invoked by the MCU Makefile.
`board-pin-mapping.pcf`	Pin constraints for iCE40UP5K SG48. All pin numbers are placeholders – edit before use.
`Makefile`	Targets: `pbi`, `cctl`, `all`, `flash-pbi`, `flash-cctl`, `flash-pbi-iceprog`, `flash-cctl-iceprog`, `doc`, `clean`.